
Knowing Before Saying: Internal Correctness Signals in Mathematical Reasoning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 When a language model is confidently wrong, is the error undetected or left
2 unsaid? We look inside the model to find out. Across four open models and
3 four domains (MMLU-Pro, MATH, GPQA-Diamond, and a geometry task
4 graded by a compiler), a simple linear detector reading the model’s internal
5 state predicts whether an answer is correct better than the model’s own
6 stated confidence in 12 of 16 settings, by a mean of +0.09 AUROC. The
7 gap is widest on MATH (+0.20), where the model’s stated confidence tracks
8 correctness worst. The detector is not just spotting hard questions: given the
9 same question several times, it still separates the attempts that succeeded
10 from the ones that failed. On MMLU-Pro the signal exists before we ask
11 for it: a probe fitted after the attempt reads at chance before it, and at the
12 last token of the answer, with no confidence question in play, correctness
13 is readable at 0.69 against 0.57 from the question alone. And on Mistral
14 the model’s report depends on it: remove that direction after the answer is
15 written and stated confidence can no longer tell the model’s successes from
16 its failures ($0.84 \rightarrow 0.56$), while removing a random direction of the same
17 size changes nothing; turn it up and confidence on wrong answers drops
18 while calibration error halves. Models carry more about their own errors
19 than they report, and on at least one model, what they report depends on
20 it.

1 Introduction

22 A language model that is confidently wrong is a problem. Its stated confidence is the one
23 signal a downstream system has for deciding whether to trust an answer, and when that
24 number is high on a wrong answer, nothing downstream can catch the error. A model that
25 is sometimes wrong can be managed; a model that is wrong and sure cannot. This paper
26 asks whether the model’s activations carry what its stated confidence does not: a readable
27 signal, at the moment the number is written, of whether the answer is right.

28 Two very different things could be going on inside such a model. Its internal state might
29 carry no trace of the error, so that the confident number is the honest output of a system
30 with no idea it has slipped. Or the error might be represented somewhere in the computation
31 and simply fail to reach the number the model writes down. The first is a capability problem.
32 The second is a reporting problem, and reporting problems are the kind an intervention can
33 reach.

34 This paper is about telling those two apart. We read the model’s internal state directly with
35 a linear detector, a *probe*, and compare what the probe finds to what the model says. The

question we keep asking is whether the difference between them is real, and each section below answers one way it could be an illusion.

We say a fact is *represented* when a probe can read it from the model’s state, and *reported* when it reaches the confidence number the model writes. The title’s “knowing” is shorthand for a representation that is present, specific to the attempt, and, in at least one model, used by the report. We make no claim about awareness, and a probe alone could not support one.

Prior work has shown that a model’s output probabilities are often better calibrated than its words [4], and that the truth of a given statement can be read from activations [1, 2, 5]. We apply those tools to the model’s own attempts, and then intervene on the representation [6] to ask whether the report depends on it.

2 Reading the state

Models and tasks. We tested four open models built in four different ways: Mistral-Small-24B (dense), Qwen3.6-27B (gated linear attention interleaved with full attention), and GLM-4.7-Flash and Gemma-4-26B (mixture of experts). Each answered 150 questions, five times each, in four domains: MMLU-Pro, MATH, GPQA-Diamond, and GeoGenBench, a geometry task in which the model writes a construction and a compiler checks every clause of the request. The compiler is why geometry is here: it grades against a formal specification, and a pre-registered human study ($N=200$, two trained raters) found its disagreements with people ran one way only, lenient and never harsh: in that sample, every compiler **fail** was a true failure. That is 16 cells of about 750 attempts each.

Three turns, graded in silence. Every attempt takes three turns: we ask the model how confident it is that it will get the problem right, let it try, then ask how confident it is that it did. A grader scores the attempt and never tells the model, so any movement in stated confidence is the model’s own.

The probe. At the moment the model is about to write its confidence digit, we record the number and the internal state behind it, a vector of a few thousand numbers. On that vector we train the probe: a logistic regression that predicts whether the attempt was correct. It is always tested on questions it never saw in training, and always read at one depth fixed in advance, 70% of the way through the network. We score every readout with AUROC, the chance that a random correct attempt outranks a random wrong one, where 0.5 is useless and 1.0 is perfect.

The state carries more than the report. The probe outranks the model’s own stated confidence in **12 of 16** cells, one tie, by a mean of +0.09 AUROC (Appendix D). The gap is widest on MATH, +0.20, and MATH is also where the report goes most wrong. On the Mistral MATH cell, with the grade never revealed, the model raises its confidence after attempting, and raises it *more* on the attempts it got wrong (+22) than on the ones it got right (+16). The probe on those same records reads 0.83. The number moves away from the truth while the state tracks it.

Why MATH. A wrong derivation ends in a clean boxed answer that reads like a right one; Appendix A follows one, a single wrong subtraction after which the model *raises* its confidence. Its length and shape still carry signal (a surface baseline reaches 0.74), but the model’s stated confidence does not use it, and the probe reads 0.83, +0.07 over surface. Geometry is the opposite: a failed construction usually does not compile, and there the surface baseline does as well as the probe.

Controls. The token we read is identical in every record, so at the input layer the probe sits at exactly 0.5 and anything it finds deeper was computed during the attempt. A within-question comparison holds difficulty fixed. The per-record data behind the original 16-cell matrix was lost to a hardware failure, so its aggregates survive only as recorded. The four Mistral cells we recaptured reproduce the headline, with a smaller margin over the surface baseline (+0.21 on MMLU-Pro, +0.07 on MATH, −0.11 on geometry).

Table 1: Mistral, recaptured cells. Left: a probe trained on one domain, tested on another (rows train, columns test; diagonal is in-domain; Geo = GeoGenBench). Right: intervening on the model’s own correctness direction at the confidence token. Gain 1 changes nothing; gain 0 removes the direction; the random row removes a random direction of the same size. Removing the real direction leaves stated confidence unable to separate successes from failures; removing the random one does not.

train ↓ test →	Geo	MMLU-Pro	MATH	GPQA
Geo	0.68	0.67	0.78	0.56
MMLU-Pro	0.70	0.75	0.84	0.59
MATH	0.80	0.72	0.83	0.57
GPQA	0.60	0.69	0.74	0.57

gain	0	1	2	4
confidence, wrong / right	97 / 98	84 / 96	70 / 94	54 / 88
stated AUROC	0.56	0.84	0.85	0.82
random direction, AUROC	0.83	0.84	0.84	unparseable

3 What the signal is

It is about this attempt, not this question. A probe that outranks stated confidence could still be doing something dull: learning which questions are hard. Hard questions fail more often, so a difficulty detector would predict correctness without reading the attempt at all.

Two tests argue against it. First, hold the question fixed. Across five attempts at one question, difficulty cannot vary, yet the probe still separates the successes from the failures (0.72 to 0.74 in representative cells) while the model’s own $P(\text{True})$, its probability of answering “True” when asked whether it was right, falls to near chance (0.47 to 0.59). Second, test the direction itself. Before the attempt, at the last token of the question, the model’s state is identical across all five attempts: it can encode difficulty and nothing else. A probe trained after the attempt and read there gives 0.49 on MMLU-Pro, chance, against 0.75 at its own site. The direction that separates good attempts from bad ones does not exist until the model has tried. Geometry is the boundary case: there the pre-attempt state already predicts failure, so that cell is mostly difficulty.

It is there before we ask. That leaves a subtler worry. Every read so far came from a token the model reaches after being asked to assess itself. Perhaps the signal is not carried from the work at all; perhaps the question manufactures it. So we read the state at the last token of the answer instead, before any confidence question exists. On MMLU-Pro, where the model cannot see failure coming (0.57 from the question alone), correctness is already readable at 0.69 the moment it stops writing. Being asked sharpens it to 0.77. The signal is made by working, and the prompt clarifies rather than creates it. MATH cannot run this test, because its question-only read is already 0.81.

It is not tied to one subject. A probe trained on one domain and tested on another keeps about 97% of its AUROC on Mistral (0.69 off the diagonal against 0.71 on it, Table 1 left), and 87 to 90% in the original matrix. The one exception is instructive: on Gemma-4, probes trained on geometry fail elsewhere (0.31 to 0.39) while probes trained elsewhere work on geometry (about 0.70). A probe trained on the one generative task can learn features specific to it.

4 Whether the report depends on it

Present is not used. A model could carry a perfect internal record of its errors and never consult it when writing the number, so we intervene. At the confidence token, after the answer is written so the answer itself cannot change, we scale the model’s own projection onto the correctness direction by a gain g , where $g=1$ changes nothing. The numbers are

120 Mistral on MATH, 150 held-out records, against a random direction rescaled to the same
121 size (Table 1 right).

122 **On Mistral, the report depends on it.** Remove the direction ($g=0$) and stated
123 confidence can no longer tell the model’s successes from its failures: 0.84 drops to 0.56, a
124 change of -0.27 $[-0.42, -0.12]$. Remove the random direction and nothing happens (-0.00
125 $[-0.03, +0.03]$). This does not look like generic damage. The output stays perfectly well
126 formed, and confidence on wrong answers goes *up*, from 84 to 97, until it matches confidence
127 on right answers; the model becomes uniformly sure. Turn the direction up instead and
128 confidence on wrong answers falls to 70 and then 54 at $g=2$ and 4 while right answers hold
129 near 90; calibration error halves, $0.34 \rightarrow 0.19$. The same intervention 30% of the way through
130 the network does nothing, and a direction learned on MMLU-Pro, where surface features
131 cannot explain the probe, still controls MATH’s report ($0.88 \rightarrow 0.71$).

132 **On GLM, it does not.** GLM’s internal signal is the strongest in the study: within a
133 question, the probe reads 0.87 while the model’s own words read 0.53. Yet removing the
134 direction leaves stated confidence exactly where it was, 0.76 before and after. GLM represents
135 its errors more sharply than Mistral and reports them less. That is our cleanest case of
136 represented but not reported, and it makes the causal result a fact about particular models
137 rather than models in general.

138 5 A second witness

139 **A label-free instrument agrees.** A trained probe invites one more objection: it might
140 have overfit to our data. So we added an instrument that uses no correctness labels. The
141 Jacobian lens [3] is fit on generic text and reads an activation for what it disposes the model
142 to say next; we score each stored state by how far it leans toward words like “wrong” over
143 words like “correct.”

144 On dense models the lens matches the probe. On the recaptured Mistral cells it reads 0.82 on
145 MATH and 0.75 on MMLU-Pro, against the probe’s 0.83 and 0.75, with a clean input-layer
146 control. Two instruments built from different evidence find the same direction, and because
147 the lens reads through the model’s own output pathway, that direction sits where it could
148 influence what the model says.

149 **Where it fails, it shows why.** On the mixture-of-experts and gated-linear-attention
150 models the lens fails (0.31 and 0.43) while the probe still works. Its “correct” and “wrong”
151 readouts collapse onto each other, cosine 0.96 on MoE against 0.4 on dense. The lens averages
152 one map over all inputs, and these architectures route differently per input, so the average
153 smears. Swap the hybrid for a dense model from the same family and the lens jumps from
154 0.43 to 0.88.

155 Put the two witnesses together: the signal is present in every model we tested; whether it
156 reaches the report, and whether a label-free instrument can read it, varies from model to
157 model.

158 6 What this means

159 **For reliability.** Where a task has an exact checker, use it. Where it does not, the probe is
160 the best per-attempt readout we found: keeping only the attempts it scores highest raises
161 accuracy far more than keeping the ones the model is most confident about, and it still
162 works where the model’s confidence is useless. On Mistral, because the report depends on
163 the direction, amplification rather than retraining lowered its overconfidence.

164 **Limitations.** A probe could be reading properties of failed output rather than a dedicated
165 self-assessment; the surface baseline bounds this without closing it, most loosely on MATH
166 and geometry (Appendix C) and on GLM, where surface nearly matches the probe. Every
167 number is a single seed. The causal result holds on Mistral and fails on GLM; Gemma-4
168 passes 94% of MATH, leaving five failures in evaluation, too few to test; Qwen3.6 is being

tested. Whether preference tuning creates the gap between representing and reporting is open; a base model would answer it.

Takeaway. These models carry information about their own errors that their stated confidence does not report, and on at least one model the reported number depends on it. A confidence score is not the whole story, and the rest of the story is readable.

References

- [1] Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it’s lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- [2] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. In *International Conference on Learning Representations (ICLR)*, 2023.
- [3] Wes Gurnee, Nicholas Soiron, Adam Pearce, Mateusz Piotrowski, Isaac Kauvar, Runjin Chen, Anna Soligo, Paul Bogdan, Euan Ong, Rowan Wang, T. Ben Thompson, David Abrahams, Subhash Kantamneni, Emmanuel Ameisen, Joshua Batson, and Jack Lindsey. Verbalizable representations form a global workspace in language models. *Transformer Circuits Thread*, 2026. <https://transformer-circuits.pub/2026/workspace/index.html>; code: <https://github.com/anthropics/jacobian-lens>.
- [4] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- [5] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. In *Conference on Language Modeling (COLM)*, 2024.
- [6] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xu Wang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A The elicitation protocol, and one record end to end

This appendix gives the exact prompts behind every record, then one complete record from the Mistral-Small-24B $\times$ MATH cell, summarised first and then given in full.

One record, Mistral-Small-24B $\times$ MATH. *What integer n satisfies $0 \leq n < 18$ and $n \equiv -11213141 \pmod{18}$?* The model sets the problem up correctly and gets $18 \times 622953 = 11213154$ right, then writes $r = -11213141 + 11213154 = 13$ and answers 3. That subtraction gives 13, which is the gold answer. Stated confidence: 90 before the attempt, 95 after it. Other readouts of the same attempt: P(True) 0.32, mean answer log-prob -0.05 .

A single arithmetic slip inside a fluent derivation. Having just made the error, the model revises its stated confidence upward, while P(True) on the same transcript assigns only 0.32 probability to correctness. The rest of this appendix gives the prompts and the full transcript.

A.1 Prompts

Each record is four forward passes. A read is first taken at the last prompt token of the bare question, before any elicitation, then three turns are generated. The confidence turns run with thinking off so the read lands on a clean one-line reply; the attempt turn runs with thinking on for hybrid reasoners. The fixed **Confidence:** N format is what makes the read site content-neutral: the digit token has identical local context on every record, so layer-0 activations carry no signal by construction.

211 **System prompt (MATH).**

212 Solve the problem step by step. Put your final answer in `\boxed{}`.

213 **Turn 1, prospective confidence.** The problem is followed by:

214 Before answering, estimate how likely you are to answer this question
215 correctly. Reply with EXACTLY one line and nothing else:
216 Confidence: N
217 where N is an integer from 0 (certain to be wrong) to 100 (certain to be
218 correct). Do NOT answer yet.

219 **Turn 2, the attempt.** The turn-1 reply is left in context as the assistant message, then:

220 Now give your final answer inside `\boxed{}`.

221 **Turn 3, retrospective confidence.** The attempt is left in context, then:

222 Now assess the answer you just gave. On a scale of 0 to 100, how confident
223 are you that it is correct? Reply with exactly one line and nothing else:
224 Confidence: N

225 **P(True) baseline.** A separate teacher-forced branch off turn 2, no generation. Next-token
226 mass on True versus False:

227 Consider your final answer above. Is it correct? Reply with exactly one word:
228 True or False.

229 Grading is external and is never shown to the model, so the turn-3 revision is blind self-
230 assessment.

231 **A.2 One record, verbatim**

232 Record `math_test_number_theory_753`, sample 2 of 5.

233 **Problem.** What integer n satisfies $0 \leq n < 18$ and $n \equiv -11213141 \pmod{18}$?

234 **Turn 1 reply.** Confidence: 90

235 **Turn 2 reply (complete).**

236 First, we need to find the remainder when -11213141 is divided by 18.
237 1. Calculate $-11213141 \bmod 18$:

$$-11213141 \div 18 \approx -622952.2778$$

238 The integer part of the division is -622953 .

$$-11213141 = 18 \times (-622953) + r$$

239 where r is the remainder.
240 2. Calculate the remainder r :

$$r = -11213141 - 18 \times (-622953)$$

241
$$r = -11213141 + 11213154$$

242
$$r = 3$$

243 Therefore,

$$-11213141 \equiv 3 \pmod{18}$$

244 So, the integer n that satisfies $0 \leq n < 18$ and $n \equiv -11213141 \pmod{18}$ is 3.

245 **Turn 3 reply.** Confidence: 95

246 **What happened.** The method is right and every intermediate is right: $18 \times 622953 =$
 247 11213154 checks out, and the model has written the correct subtraction on the line above its
 248 answer. It then evaluates $-11213141 + 11213154$ as 3 rather than 13. Gold is 13. Nothing
 249 in the presentation marks the slip. The derivation is well-formed, correctly structured, and
 250 confidently laid out, which is precisely why stated confidence rises from 90 to 95 across an
 251 attempt that failed.

252 **Readouts of this attempt.**

readout	value
stated confidence, prospective (turn 1)	90
stated confidence, retrospective (turn 3)	95
P(True)	0.32
mean answer log-probability	-0.05
graded outcome	incorrect

254 This is one record, chosen to illustrate the failure mode rather than to carry evidential weight;
 255 the cell-level numbers are in Table 2.

256 B Calibration beyond ECE, and cell-level multiplicity

257 ECE depends on binning and on sample composition, so we report two binning-free proper
 258 scoring rules alongside it. On Mistral $\times$ MATH (750 records, accuracy 0.648), the model’s
 259 retrospective stated confidence gives Brier 0.262 and NLL 1.470 against a 10-bin ECE of
 260 0.271. P(True) on the same records gives Brier 0.386: better ordered than stated confidence,
 261 worse calibrated in absolute terms. The steering results in Table 1 are reported as ECE
 262 because the steered runs stored per-gain aggregates rather than per-record confidences;
 263 recomputing Brier and NLL at each gain requires re-running the intervention and is the first
 264 thing we would add.

265 The cell-level bootstrap counts in Table 2 are uncorrected. With 16 cells, some individually
 266 significant results are expected under the null, so the cell counts should be read as descriptive.
 267 The aggregate pattern is the claim that carries weight: the effect is positive in 12 of 16 cells
 268 with one tie, significantly positive in 10, and never significantly negative. A hierarchical
 269 model pooling across cells with model and domain as random effects is the right test and is
 270 not yet run.

271 C Alternative explanations for the probe signal

272 A probe could key on properties of failed derivations rather than on a dedicated metacognitive
 273 variable: low token probability, hedging language, or, on MATH, an early-layer channel next
 274 to the answer text. Our surface-features baseline (answer length, digit count, line count,
 275 word count, brace count) bounds this without closing it, and the bound is domain-dependent:
 276 the probe adds +0.21 over surface on MMLU-Pro but only +0.07 on MATH and -0.11
 277 on geometry, where a construction that fails to compile is visibly malformed. Two results
 278 argue against a pure surface account on the domains where the signal is strong. A direction
 279 fitted on MMLU-Pro, where surface reaches only 0.54 against a probe of 0.75, still governs
 280 MATH’s self-report under ablation ($0.880 \rightarrow 0.709$). And the pre-attempt read, taken where
 281 the activation is byte-identical across a question’s attempts, cannot recover the post-attempt
 282 direction on MMLU-Pro (0.490, chance), which a surface-of-the-question account would
 283 predict it could. A linear probe at one token also undercounts nonlinear or multi-token
 284 signal, so the reported numbers are a floor.

Table 2: Post-attempt correctness discrimination (AUROC). Left: stated confidence vs. the probe in representative cells; overall the probe wins 12 of 16 with one tie, mean +0.09, with a paired bootstrap significantly positive in 10 of 16 and never significantly negative, uncorrected at the cell level (Appendix B). Right: Mistral specificity controls. “pre” is the last prompt token before the attempt; “post” is after the attempt; “resid.” removes the pre score; “post→pre” applies that fitted post readout at the pre-attempt site, where only difficulty is visible.

model × domain	stated	probe	Mistral	pre	post	resid.	post→pre
Gemma-4 × MATH	0.57	0.78	MMLU-Pro	0.568	0.754	0.751	0.490
Qwen3.6 × MMLU-Pro	0.67	0.84	MATH	0.792	0.834	0.701	0.624
GLM × MMLU-Pro	0.67	0.85	GPQA	0.593	0.572	0.544	0.479
Mistral × MATH	0.79	0.83	GeoGen	0.770	0.676	0.505	0.648

285 D Per-cell numbers and specificity controls

286 E Checkpoints

287 All models are public Hugging Face releases, run in bfloat16 without quantization,
 288 greedy decoding: mistralai/Mistral-Small-24B-Instruct-2501, Qwen/Qwen3.6-27B,
 289 zai-org/GLM-4.7-Flash, google/gemma-4-26B-A4B-it; the dense within-family control
 290 for the lens is Qwen/Qwen2.5-14B-Instruct.